

Моделирование графов атак

**Дисциплина: Методы математического моделирования в
кибербезопасности**

Верниковская Екатерина Андреевна

Содержание

1	Моделирование графов атак	5
1.1	Что такое граф атак	5
1.2	Как строится граф	5
1.3	Какие алгоритмы используются	5
1.4	Что анализируется	6
1.5	Оснвные коды	6
1.6	Ядро моделирования графа атак	7
1.7	Однократный запуск эксперимента	12
1.8	Визуализация и анализ графа	15
1.9	Исследование масштабируемости	19
1.10	Параметрическое исследование	22

Список иллюстраций

Список таблиц

1 Моделирование графов атак

1.1 Что такое граф атак

Граф атак - это ориентированный граф, где вершины - сетевые узлы (хосты, серверы), а рёбра - возможность атаки с одного узла на другой (через уязвимость, доверительные отношения, сетевой доступ).

Граф позволяет:

- найти все возможные пути от атакующего к цели
- выявить критические узлы
- оценить наиболее вероятный путь атаки

1.2 Как строится граф

Граф создаётся комбинацией:

- случайных рёбер - с заданной вероятностью
- доверительных отношений - жёстко заданные связи (админский доступ)
- весов рёбер - вероятность успешной атаки (например, из CVSS-оценок)

1.3 Какие алгоритмы используются

- Поиск всех путей — рекурсивный DFS от источника к цели

- Наиболее вероятный путь — через логарифмирование вероятностей и алгоритм Дейкстры
- Метрики центральности — in-degree (сколько атак ведёт в узел), out-degree, PageRank (рекурсивная важность узла)

1.4 Что анализируется

1. Количество путей атаки — чем больше, тем выше риск
2. Наиболее вероятный путь — приоритет для защиты
3. Критические узлы — с высокой входящей степенью или PageRank
4. Масштабируемость — время поиска путей растёт экспоненциально с размером сети
5. Влияние плотности рёбер — чем связнее сеть, тем больше путей и их средняя длина

1.5 Основные коды

1. Файл `rc/attack_graph.jl` - ядро моделирования, содержащее функции для построения графа, поиска всех путей атаки, расчёта метрик центральности (in-degree, PageRank и др.) и нахождения наиболее вероятного пути через алгоритм Дейкстры.
2. Файл `scripts/ag_run_experiment.jl` - однократный запуск эксперимента: строит граф с заданными параметрами, находит все пути и сохраняет результаты в JLD2-файл.
3. Файл `scripts/ag_analyze.jl` - загружает сохранённые данные, визуализирует граф с цветовой индикацией по PageRank и выводит статистику (количество путей, топ узлов по in-degree и PageRank).

4. Файл `scripts/ag_convergence.jl` - исследует масштабируемость алгоритма, измеряя время поиска путей и их количество в зависимости от размера сети.
5. Файл `scripts/parameter_sweep.jl` - параметрическое исследование, анализирующее, как плотность рёбер влияет на количество путей атаки и их среднюю длину.

1.6 Ядро моделирования графа атак

```
using Graphs, LinearAlgebra
```

```
"""
```

```
build_attack_graph(n, edge_prob, vulnerabilities, trust_relations)
```

```
Создаёт ориентированный граф атак с заданными параметрами.
```

```
"""
```

```
function build_attack_graph(n, edge_prob, vulnerabilities,
```

```
    trust_relations)
```

```
    g = SimpleDiGraph(n)
```

```
    # случайные рёбра
```

```
    for i = 1:n, j = 1:n
```

```
        if i != j && rand() < edge_prob
```

```
            add_edge!(g, i, j)
```

```
        end
```

```
    end
```

```
    # доверительные отношения
```

```
    for (u, v) in trust_relations
```

```
        add_edge!(g, u, v)
```

```
    end
```

```

        return g
    end

    """
    find_all_paths(g, source, target)
    Рекурсивный поиск всех простых путей от source до target.
    """

    function find_all_paths(g, source, target)
        paths = []
        function dfs(current, path)
            if current == target
                push!(paths, copy(path))
                return
            end
            for neighbor in outneighbors(g, current)
                if !(neighbor in path)
                    push!(path, neighbor)
                    dfs(neighbor, path)
                    pop!(path)
                end
            end
        end
        dfs(source, [source])
        return paths
    end

    """
    compute_centrality_metrics(g)
    Вычисляет основные метрики центральности.

```



```

"""
function compute centrality_metrics(g)
    indeg = indegree(g)
    outdeg = outdegree(g)
    betweenness = betweenness centrality(g)
    closeness = closeness centrality(g)
    pagerank = simple_pagerank(g)
    # используем нашу реализацию
    return Dict(
        :in_degree => indeg,
        :out_degree => outdeg,
        :betweenness => betweenness,
        :closeness => closeness,
        :pagerank => pagerank,
    )
end

"""
assign_edge_weights(g, cvss_scores)
Присваивает каждому ребру вес на основе CVSS-оценок (по умолчанию
↪ 0.5).
"""

function assign_edge_weights(g, cvss_scores)
    weights = Dict{Edge,Float64}()
    for e in edges(g)
        u, v = src(e), dst(e)
        key = (u, v)
        weight = get(cvss_scores, key, 0.5)
        weights[e] = weight
    end
end

```

```

    end
    return weights
end

"""
most_likely_path(g, source, target, weights)
Находит путь с максимальным произведением вероятностей.
"""

function most_likely_path(g, source, target, weights)
    n = nv(g)
    # Создаём матрицу весов размером n×n, заполненную
    ↪ бесконечностью
    distmx = fill(Inf, n, n)
    for e in edges(g)
        u, v = src(e), dst(e)
        w = weights[e] # вероятность успешной атаки на ребре
        distmx[u, v] = -log(w) # вес для Дейкстры (логарифм
        ↪ обратной вероятности)
    end
    state = dijkstra_shortest_paths(g, source, distmx)
    dist = state.dists
    parents = state.parents
    if dist[target] == Inf
        return [], 0.0
    end
    # Восстанавливаем путь
    path = Int[]
    current = target
    while current != source

```

```

        push!(path, current)
        current = parents[current]
    end
    push!(path, source)
    reverse!(path)
    prob = exp(-dist[target]) ## преобразуем обратно в вероятность
    return path, prob
end

"""
simple_pagerank(g;  $\alpha=0.85$ , max_iter=100, tol=1e-6)
Простая реализация PageRank для ориентированного графа.
"""

function simple_pagerank(g;  $\alpha = 0.85$ , max_iter = 100, tol = 1e-6)
    n = nv(g)
    n == 0 && return Float64[]
    pr = fill(1.0 / n, n)
    for _ = 1:max_iter
        pr_new = fill((1- $\alpha$ )/n, n)
        for i = 1:n
            outdeg = outdegree(g, i)
            if outdeg > 0
                for j in outneighbors(g, i)
                    pr_new[j] +=  $\alpha$  * pr[i] / outdeg
                end
            else
                # телепортация из узлов без исходящих рёбер
                for j = 1:n

```

```

                pr_new[j] +=  $\alpha$  * pr[i] / n
            end
        end
    end
    diff = maximum(abs.(pr_new - pr))
    pr = pr_new
    if diff < tol
        break
    end
end
return pr
end

```

simple_pagerank (generic function with 1 method)

1.7 Однократный запуск эксперимента

```

using Pkg
Pkg.activate("../project")
using DrWatson
@quickactivate "project"
using Graphs, JLD2, Random

include(scrdir("attack_graph.jl"))

params = Dict(
    :n ⇒ 20,
    :edge_prob ⇒ 0.2,

```

```

:source ⇒ 1,
:target ⇒ 20,
:cvss_scores ⇒ Dict{(1, 3) ⇒ 0.9, (2, 5) ⇒ 0.7, (3, 8) ⇒
    ↪ 0.8, (5, 20) ⇒ 0.95),
:trust_relations ⇒ [(4, 6), (6, 10), (10, 15)],
)

```

```

filename = datadir("attack_graph", savename(params, "jld2"))
mkpath(datadir("attack_graph"))

```

```

if isfile(filename)
    @load filename data
    println("Данные загружены из $filename")

```

```

else
    g = build_attack_graph(
        params[:n],
        params[:edge_prob],
        params[:cvss_scores],
        params[:trust_relations],
    )

```

```

paths = find_all_paths(g, params[:source], params[:target])
metrics = compute_centrality_metrics(g)
weights = assign_edge_weights(g, params[:cvss_scores])
likely_path, prob = most_likely_path(g, params[:source],
    ↪ params[:target], weights)

```

```

data = Dict(
    :graph ⇒ g,

```

```

        :paths ⇒ paths,
        :metrics ⇒ metrics,
        :weights ⇒ weights,
        :likely_path ⇒ likely_path,
        :probability ⇒ prob,
    )

    @save filename data params
    println("Результаты сохранены в $filename")
end

println("Количество путей атаки: ", length(data[:paths]))
println("Наиболее вероятный путь: ", data[:likely_path])
println("Вероятность успеха: ", data[:probability])

```

Activating project at `~/work/study/2025-2026/Кибербезопасность/security-modeling/labs/lab02/project`

```

└ Warning: DrWatson could not find find a project file by recursively checking
  | (given dir: /home/eavernikovskaya)
└ @ DrWatson ~/.julia/packages/DrWatson/2QF5p/src/project_setup.jl:84

```

Данные загружены из /home/eavernikovskaya/work/study/2025-2026/Кибербезопасность/security-modeling/labs/lab02/project/data/attack_graph/edge_prob=0.2_n=20_source=1_target=2

Количество путей атаки: 3841

Наиболее вероятный путь: [1, 19, 8, 18, 20]

Вероятность успеха: 0.0625

1.8 Визуализация и анализ графа

```
using Pkg
Pkg.activate("../project")
using DrWatson
@quickactivate "project"
using Graphs, JLD2, Plots, GraphRecipes
default(fmt = :png)

include(scrdir("attack_graph.jl"))

params = Dict{:n ⇒ 20, :edge_prob ⇒ 0.2, :source ⇒ 1, :target
    ↪ ⇒ 20}
filename = datadir("attack_graph", savename(params, "jld2"))
@load filename data

g = data[:graph]
metrics = data[:metrics]

pagerank = metrics[:pagerank]
norm_rank = (pagerank .- minimum(pagerank)) ./ (maximum(pagerank)
    ↪ - minimum(pagerank))
colors = [cgrad(:RdYlGn, rev = true)[norm_rank[i]] for i =
    ↪ 1:nv(g)]

plot_graph = graphplot(
    g,
    nodeshape = :circle,
```

```

    curves = false,
    linecolor = :black,
    nodecolor = colors,
    nodelabel = 1:nv(g),
    title = "Граф атак (цвет = PageRank)",
    size = (800, 600),
)

savefig(plot_graph, plotsdir("attack_graph.png"))
println("График сохранён в ", plotsdir("attack_graph.png"))

display(plot_graph)

println("=== Анализ графа атак ===")
println("Количество узлов: ", nv(g))
println("Количество рёбер: ", ne(g))
println("Количество путей от ", params[:source], " к ",
    ↪ params[:target], ": ", length(data[:paths]),)

if !isempty(data[:paths])
    println("Длины путей: ", [length(p) for p in data[:paths]])
    println("Наиболее вероятный путь: ", data[:likely_path])
end

println("\nТоп-5 узлов по in-degree:")
top_indeg = sortperm(metrics[:in_degree], rev = true)[1:5]

for i in top_indeg
    println(" Узел $i: in-degree = $(metrics[:in_degree][i])")
end

```


end

```
println("\nТоп-5 узлов по PageRank:")
```

```
top_pr = sortperm(metrics[:pagerank], rev = true)[1:5]
```

```
for i in top_pr
```

```
    println(" Узел $i: PageRank = $(round(metrics[:pagerank][i],  
    ↪ digits=4))")
```

```
end
```

График сохранён в /home/eavernikovskaya/work/study/2025-2026/Кибербезопасность/
modeling/labs/lab02/project/plots/attack_graph.png

```
Activating project at `~/work/study/2025-2026/Кибербезопасность/security-  
modeling/labs/lab02/project`
```

```
└ Warning: DrWatson could not find find a project file by recursively checking  
  | (given dir: /home/eavernikovskaya)
```

```
└ @ DrWatson ~/.julia/packages/DrWatson/2QF5p/src/project_setup.jl:84
```

A complex directed graph with 20 nodes and many edges. The nodes are colored in a repeating pattern of green, red, yellow, and orange. The graph is highly interconnected, with many edges pointing in various directions, suggesting a complex network structure.

Наиболее вероятный путь: [1, 19, 8, 18, 20]

Узел 19: in-degree = 6

Топ-5 узлов по PageRank:

Узел 13: PageRank = 0.0952

Узел 8: PageRank = 0.0837

Узел 3: PageRank = 0.0828

Узел 18: PageRank = 0.0826

Узел 10: PageRank = 0.0776

1.9 Исследование масштабируемости

```
using Pkg
Pkg.activate("../project")
using DrWatson
@quickactivate"project"

using Graphs
using Plots
using JLD2
using Random
default(fmt = :png)

include(sourcedir("attack_graph.jl"))

sizes = [10, 15, 20, 21, 22, 23, 24, 25]
time_vals = Float64[]
path_counts = Int[]

Random.seed!(123)
```

```

for n in sizes
    println("Размер сети: $n")
    g = build_attack_graph(n, 0.2, Dict(), [])
    t = @elapsed paths = find_all_paths(g, 1, n)
    push!(time_vals, t)
    push!(path_counts, length(paths))
    println(" Время: $(round(t, digits=3)) с, путей:
        ↪ $(length(paths))")

```

```

end

```

```

p1 = plot(
    sizes,
    time_vals,
    marker = :circle,
    xlabel = "Число узлов",
    ylabel = "Время (с)",
    title = "Время поиска всех путей",
    legend = false,
)

```

```

p2 = plot(
    sizes,
    path_counts,
    marker = :circle,
    xlabel = "Число узлов",
    ylabel = "Количество путей",
    title = "Количество путей атаки",
    legend = false,
)

```

```

)

combined = plot(p1, p2, layout = (2, 1), size = (800, 600))
savefig(combined, plotsdir("convergence.png"))

display(combined)

mkpath(datadir("convergence"))
data = Dict{:sizes => sizes, :times => time_vals, :path_counts =>
    path_counts}
@save datadir("convergence", "convergence_data.jld2") data

```

Размер сети: 10

Время: 0.034 с, путей: 1

Activating project at `~/work/study/2025-2026/Кибербезопасность/security-modeling/labs/lab02/project`

```

└ Warning: DrWatson could not find find a project file by recursively checking
  | (given dir: /home/eavernikovskaya)
└ @ DrWatson ~/.julia/packages/DrWatson/2QF5p/src/project_setup.jl:84

```

Размер сети: 15

Время: 0.0 с, путей: 842

Размер сети: 20

Время: 0.0 с, путей: 817

Размер сети: 21

Время: 0.017 с, путей: 58966

Размер сети: 22

Время: 0.185 с, путей: 355669

Размер сети: 23

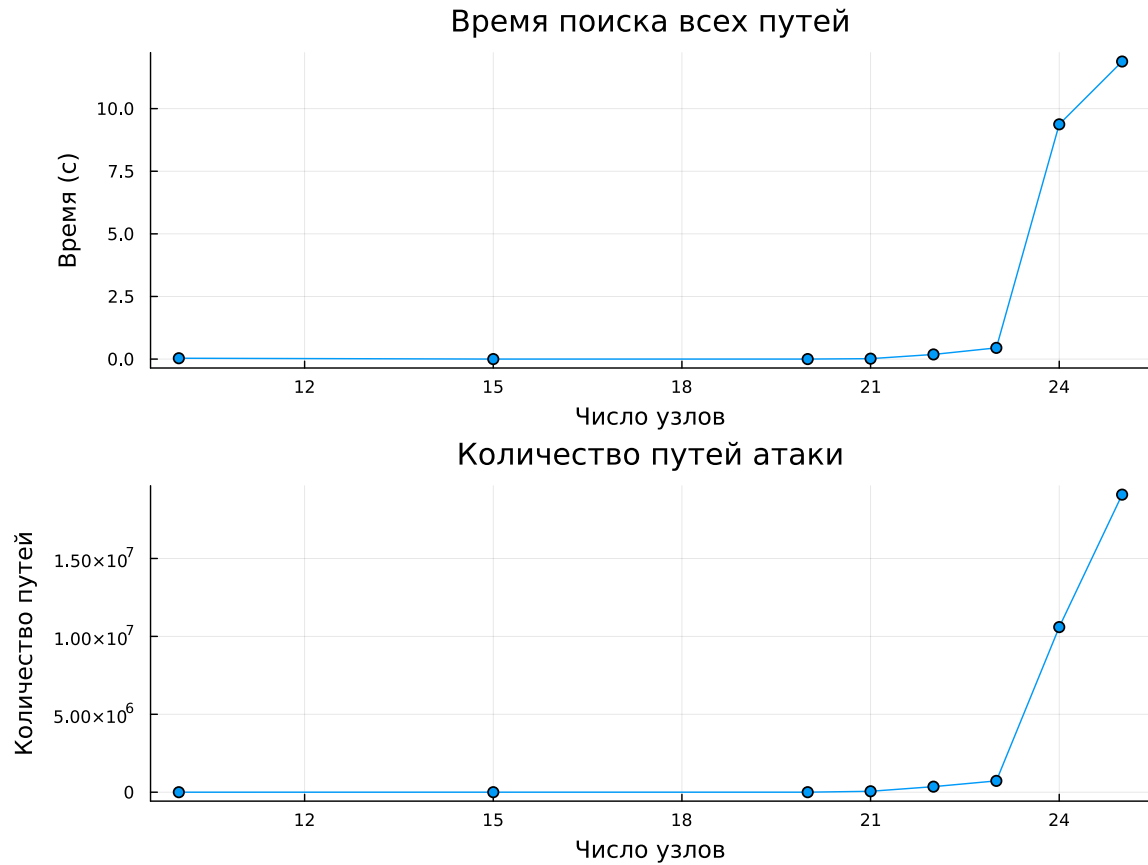
Время: 0.447 с, путей: 726135

Размер сети: 24

Время: 9.373 с, путей: 10596200

Размер сети: 25

Время: 11.881 с, путей: 19097246



1.10 Параметрическое исследование

```
using Pkg
Pkg.activate("../project")
using DrWatson
@quickactivate"project"
```

```

using Graphs, Random, CSV, DataFrames, Plots, StatsBase
default(fmt = :png)

include(sourcedir("attack_graph.jl"))

edge_probs = 0.1:0.1:0.9
n_nodes = 13
source = 1
target = n_nodes

results = []
Random.seed!(123)

for p in edge_probs
    println("Плотность рёбер: $p")
    g = build_attack_graph(n_nodes, p, Dict(), [])
    paths = find_all_paths(g, source, target)
    metrics = compute_centrality_metrics(g)
    max_indeg = maximum(metrics[:in_degree])
    avg_path_len = isempty(paths) ? 0.0 : mean(length.(paths))
    push!(
        results,
        (p = p, paths = length(paths), max_indeg = max_indeg,
         ↪ avg_len = avg_path_len),
    )
end

mkpath(datadir("parameter_sweep"))
df = DataFrame(results)

```

```
CSV.write(datadir("parameter_sweep", "results.csv"), df)
```

```
p1 = plot(  
  edge_probs,  
  [r.paths for r in results],  
  marker = :circle,  
  xlabel = "Плотность рёбер",  
  ylabel = "Количество путей",  
  label = "Пути",  
)
```

```
p2 = plot(  
  edge_probs,  
  [r.avg_len for r in results],  
  marker = :circle,  
  xlabel = "Плотность рёбер",  
  ylabel = "Средняя длина пути",  
  label = "Средняя длина",  
)
```

```
combined = plot(p1, p2, layout = (2, 1), size = (800, 600))  
savefig(combined, plotsdir("parameter_sweep.png"))
```

```
display(combined)
```

Плотность рёбер: 0.1

Плотность рёбер: 0.2

Activating project at `~/work/study/2025-2026/Кибербезопасность/security-modeling/labs/lab02/project`


```
Warning: DrWatson could not find find a project file by recursively checking  
(given dir: /home/eavernikovskaya)  
@ DrWatson ~/.julia/packages/DrWatson/2QF5p/src/project_setup.jl:84
```

Плотность рёбер: 0.3

Плотность рёбер: 0.4

Плотность рёбер: 0.5

Плотность рёбер: 0.6

Плотность рёбер: 0.7

Плотность рёбер: 0.8

Плотность рёбер: 0.9

